

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 12

Microframeworks und Testing

12.1 Einführung in Microframeworks

Wie bereits in Kapitel 5 erläutert wurde, hat sich die Art und Weise, mit PHP-Code zu arbeiten, in den letzten Jahren stark verändert. Statt Do It Yourself (DIY) Lösungen existieren große Frameworks, die einen großen Funktionsumfang mit sich bringen, oder auch Komponenten, die genau eine Aufgabe haben und für sehr spezifische Anwendungsfälle eingesetzt werden können. Dazwischen existiert jedoch noch eine weitere Ebene, die bis jetzt noch nicht erwähnt wurde: Microframeworks.

12.1.1 Definition

Ein *Microframework* ist ein minimalistisches Webframework, das im Vergleich zu den sogenannten Full-Stack-Frameworks (wie etwa Symfony, Laravel etc.) einen wesentlich geringeren Funktionsumfang bietet. Dieser wird jedoch in vielen Fällen gar nicht benötigt, etwa wenn nur ein kleines Projekt oder eine REST-API (im Sinne eines Web-Services) umgesetzt werden sollen.

Sogenannte Full-Stack Frameworks, im Englischen auch oft „Kitchen Sink Solution“ genannt (vom Ausspruch „everything but the kitchen sink“, also etwas, das so ziemlich alles umfasst) bieten in ihrem Umfang Lösungen für die verschiedensten Bereiche einer Webseite an. Dazu gehören u. a. Routing (erläutert in Kapitel 6), Logging (das Mitspeichern von Log-Informationen, ebenfalls in Kapitel 6 erwähnt), Datenbank-Management, Komponenten zur Authentifizierung (Login), Abstraktion des Dateisystems und noch viele mehr. Viele dieser Komponenten werden möglicherweise für ein Projekt gar nicht benötigt, das Full-Stack Framework sieht sich jedoch als Werkzeugkasten, der die Dinge einfach anbietet, auch wenn Sie nicht benötigt werden. Abbildung 12.1 stellt die Struktur eines solchen Frameworks schematisch dar.

Ein Microframework im Gegenzug enthält nur die notwendigsten Komponenten. Dies ist in der Regel das Routing, alle anderen Dinge sind nicht enthalten. Sie können jedoch als Dependencies (d. h. als Komponenten) ganz einfach nachinstalliert werden und werden dabei vom Microframework in einem eigenen Container verwaltet. Zusätzlich unterstützen viele Microframeworks noch sogenannte Middleware. Dies sind ebenfalls Komponenten, deren Aufgaben jedoch explizit darin bestehen, die Request- und Response-Objekte (die Anfrage und Ergebnis beinhalten) zu verändern. Abbildung 12.2

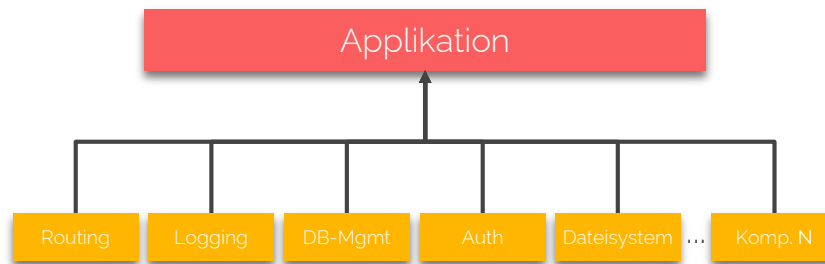


Abbildung 12.1: Ein Full-Stack Framework beinhaltet eine Vielzahl von verschiedenen Komponenten. Abbildung basierend auf [2].

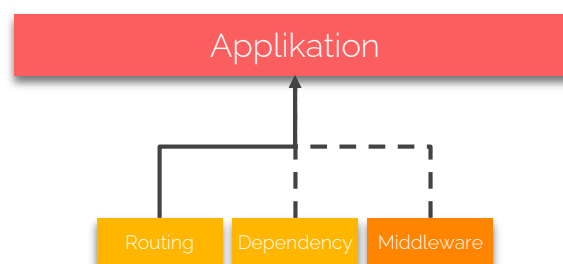


Abbildung 12.2: Das Microframework enthält im Kern nur das Routing. Andere Dependencies oder Middleware sind optional. Abbildung basierend auf [2].

stellt die grundsätzliche Architektur eines Microframeworks dar.

Anders ausgedrückt: Ein Microframework enthält von sich aus nur das Nötigste, um eine Webapplikation zu erstellen: das Empfangen eines HTTP-Requests, die Weiterleitung (Routing) dieses Requests an eine Callback-Methode (die dann die Response erstellt) und Rückgabe der HTTP-Response.

12.1.2 Vertreter

Es gibt eine Reihe von Microframeworks in PHP, diese erfüllen alle denselben Zweck, nämlich eine einfache Routing-Basis für eine Web-Applikation zu sein, ihr Funktionsumfang unterscheidet sich aber mitunter leicht. Im Folgenden werden einige Vertreter kurz beschrieben.

Slim Framework

Das *Slim Framework* wird in dieser Vorlesung im Detail in Abschnitt 12.2 behandelt.

Fat-Free Framework

Das *Fat-Free Framework*¹ (kurz F3) existiert seit 2009 und ist somit eines der ältesten Microframeworks. Es enthält Routing, Caching und Unterstützung für mehrsprachige

¹<https://fatfreeframework.com/>

Applikationen. Weitere Funktionalitäten können über eigene Plug-ins nachgerüstet werden, dazu zählen eine Template-Engine, ein Unit-Testing Toolkit oder auch Datenbank-Anbindungen. Der Quellcode des Frameworks hat nur 90 kB, was F3 zu einem der kleinsten Frameworks macht.

Mezzio

*Mezzio*² ist ein Microframework, das auf den Komponenten des Laminas (vormals Zend) Frameworks aufbaut und sich vor allem als Alternative für Entwickler*innen sieht, die mit diesem Full-Stack Framework bereits Erfahrungen haben. Umgekehrt lassen sich auch Anwendungen, die mit Mezzio erstellt wurden, leicht auf Laminas upgraden, falls dies erforderlich wird. Mezzio beinhaltet neben dem Routing Komponenten für Authentifikation und Autorisierung sowie Sessions.

Flight

*Flight*³ ist ein weiteres Microframework, das auf eine kompakte Codebasis setzt. Es besteht im Kern aus einem Router, besitzt jedoch auch Möglichkeiten zum Rendern des Outputs mittels PHP. Ebenfalls enthalten ist eine Möglichkeit des Cachings und sogenannte Filter, die vor und nach Funktionen aufgerufen werden können und Parameter und Output verändern.

12.2 Slim Framework

Das *Slim Framework*⁴ ist ein PHP Microframework zum effizienten Erstellen von Webapplikationen und APIs (Schnittstellen). Es besteht im Kern aus folgenden Funktionalitäten bzw. unterstützt die folgenden Dinge:

- **HTTP-Routing:** Der Router von Slim erlaubt es, Callback-Methoden zu spezifischen HTTP-Methoden (GET, POST etc.) und entsprechenden URL-Mustern zu erstellen, die aufgerufen werden, wenn der entsprechende Request die Webapplikation erreicht. Der Router unterstützt dabei fortgeschrittene Funktionalitäten wie Parameter oder Pattern-Matching.
- **Middleware:** Middleware-Komponenten können verwendet werden, um Request- und Response-Objekte zu verändern, z. B. um Autorisierung zu überprüfen, bevor ein Request beantwortet wird.
- **PSR-7 Unterstützung:** PSR-7⁵ ist ein Standard der PHP-FIG und definiert, wie Request- und Response-Objekte (HTTP-Messages) funktionieren sollen, um dies zu vereinheitlichen. Slim unterstützt dabei alle PSR-7-kompatiblen HTTP-Message-Implementierungen.
- **Dependency Injection:** Werden weitere Dependencies, d. h. Komponenten benötigt, können diese in einen Container geladen und damit verwaltet werden. Sie

²<https://docs.mezzio.dev/>

³<https://docs.flightphp.com/>

⁴<https://www.slimframework.com/>

⁵<https://www.php-fig.org/psr/psr-7/>

werden erst geladen, wenn sie benötigt werden. Jede Implementierung, die dem PSR-11-Standard für Container⁶ folgt, wird dabei unterstützt.

Die aktuelle Version ist 4.15.2 vom 22. Mai 2026.

12.2.1 Technische Voraussetzungen

Slim Framework benötigt wie alle PHP-Anwendungen einen Webserver, z. B. Apache oder nginx. Da die Routing-Funktionalität jedoch darauf baut, dass URLs aufgerufen werden, die nicht existieren (d. h. die auf kein existierendes Verzeichnis am Server zeigen), ist es nötig, dass der Webserver sogenanntes URL-Rewriting unterstützt. Dabei werden URL-Angaben, die am Webserver nicht existieren, über ein Set an Regeln zurück an die Hauptdatei (z. B. `index.php`) geleitet. Diese Datei kümmert sich dann darum, die angegebene Route zu verarbeiten und etwas Sinnvolles (oder auch eine Fehlerseite) anzuzeigen. Dies ist das bereits in Kapitel 6 angesprochene *Front-Controller-Pattern*.

Wie beim `fhoee/router` muss für den Apache Webserver das Modul `mod_rewrite` aktiviert sein und für das Web-Verzeichnis die Direktive `AllowOverride All` gesetzt sein. Dies erlaubt, dass die Datei `.htaccess` erstellt werden kann, die die entsprechenden Rewrite-Regeln enthält.

Die `.htaccess`-Datei für Slim gleicht der des `fhoee/router`:

```
1 <IfModule mod_rewrite.c>
2   RewriteEngine On
3
4   RewriteCond %{REQUEST_URI}::$1 ^(/.+)/(.*):\:2$
5   RewriteRule ^(.*) - [E=BASE:%1]
6
7   RewriteCond %{REQUEST_FILENAME} !-f
8   RewriteRule ^ index.php [QSA,L]
9 </IfModule>
```

Zunächst wird überprüft, ob `mod_rewrite` aktiv ist und falls ja, die Rewrite-Engine aktiviert. In den Zeilen 4 und 5 wird dann das Basisverzeichnis ermittelt, falls das Projekt nicht im Root des Webserver liegt. In Zeile 7 wird schließlich als Bedingung abgefragt, dass die Datei nicht existiert und in diesem Fall dann `index.php` in Zeile 8 aufgerufen. Das Argument `QSA` gibt an, dass GET-Parameters angehängt werden, `L` stoppt die Verarbeitung von Regeln nach der aktuellen.

Die Konfiguration der Rewrite-Rule für nginx kann der Slim-Dokumentation⁷ entnommen werden. Ebenso findet sich dort die Möglichkeit, mit dem eingebauten PHP-Webserver zu testen.

12.2.2 Hello World-Beispiel

Um eine Applikation mit Slim zu erstellen, bietet sich der Slim-Skeleton⁸ an, der – ähnlich wie der `fhoee/router-skeleton` – eine simple Ausgangsbasis darstellt. Für dieses Hello World-Beispiel wird jedoch komplett von vorne begonnen.

⁶<https://www.php-fig.org/psr/psr-11/>

⁷<https://www.slimframework.com/docs/v4/start/web-servers.html>

⁸<https://github.com/slimphp/Slim-Skeleton>

Verzeichnisstruktur

Zunächst wird ein Verzeichnis für das gesamte Projekt – in diesem Fall **helloworld** angelegt. Anschließend noch ein Unterverzeichnis **public**, in dem sich die aufzurufende(n) Datei(en), nämlich **index.php** und auch **.htaccess** befinden.

Einbinden von Slim

Danach wird im Verzeichnis **helloworld** Slim mittels Composer installiert:

```
1 $ composer require slim/slim
```

Dies erzeugt **composer.json**, **composer.lock** und das **vendor**-Verzeichnis mit den von Slim benötigten Dependencies. Es empfiehlt sich, stets die aktuelle Version des Frameworks verwenden (aktuell 4.15.2), da diese laufend Fehler- und Sicherheitskorrekturen erhält.

Eine PSR-7 Implementierung einbinden

PSR-7 ist der Standard, der HTTP-Messages (Requests und Responses) als Interfaces definiert. Slim baut auf diesem Standard auf und benötigt daher eine konkrete Implementierung dieser Interfaces. Verwendet werden kann dabei eine Reihe von Komponenten, einige davon sind in der Slim-Dokumentation⁹ aufgelistet. Slim bietet selbst eine Implementierung an, nämlich **slim/psr7**, die mit Composer im nächsten Schritt eingebunden wird:

```
1 $ composer require slim/psr7
```

Die Applikation erzeugen

In **public** wird zunächst eine **.htaccess**-Datei mit dem Inhalt aus Abschnitt 12.2.1 erzeugt.

Anschließend wird die Hauptdatei **index.php** im selben Verzeichnis angelegt und mit folgendem Inhalt befüllt:

```
1 use Slim\Factory\AppFactory;
2 use Psr\Http\Message\ResponseInterface as Response;
3 use Psr\Http\Message\ServerRequestInterface as Request;
4
5 require __DIR__ . '/../vendor/autoload.php';
6
7 $app = AppFactory::create();
8
9 $app->setBasePath("/wet2vo-examples/vo12/slim-framework/helloworld/public");
10 $app->addErrorMiddleware(true, false, false);
11
12 $app->get("/", function (Request $request, Response $response, array $args) {
13     $response->getBody()->write("Hello world!");
14     return $response;
15 });
16
17 $app->run();
```

⁹<https://www.slimframework.com/docs/v4/start/installation.html>

Importe: Um den Code lesbarer zu machen, werden zu Beginn die benötigten Namespaces mittels `use`-Direktive angegeben. Für Request und Response werden dabei die PSR-7-Interfaces (`ServerRequestInterface` bzw. `ResponseInterface`) importiert, so dass nicht gegen eine konkrete Implementierung, sondern gegen das Interface programmiert wird (lose Kopplung).

Mit dem Schlüsselwort `as` wird dabei ein *Alias* für den Namen des Interfaces vergeben: `use Psr\Http\Message\ResponseInterface as Response` macht den vollqualifizierten Namen unter der Kurzbezeichnung `Response` verfügbar. Ein Alias ist rein lokal und ändert nichts an der Klasse selbst – er dient nur dazu, lange Namen abzukürzen oder Namenskonflikte zwischen gleichnamigen Klassen aus unterschiedlichen Namespaces aufzulösen. In den folgenden Beispielen kann dadurch die kürzere Schreibweise `Request` bzw. `Response` in den Methoden-Signaturen verwendet werden. Im Anschluss folgt die Einbindung des Composer-Autoloaders.

App-Objekt erzeugen: Der Einstiegspunkt in eine Slim-Applikation ist ein Objekt der Klasse `\Slim\App`. Die Applikation verwaltet primär die Routen, d.h. die möglichen URL-Pfade, unter denen Inhalte zurückgeliefert werden und beinhaltet den Container, der Dependencies und auch Einstellungen verwaltet. Ein App-Objekt wird jedoch nicht direkt mit `new` angelegt, sondern über eine sogenannte Factory (Deutsch auch Fabrik-methode genannt) erzeugt.

Base-Path angeben: Soll eine Slim-Applikation in einem Unterverzeichnis des Document-Roots laufen, muss dieses – wie auch schon vom `fhooe/router` bekannt – als Base-Path angegeben werden. Angenommen, das Document-Root ist (wie unter vielen Linux-Distributionen üblich) unter `var/www/html` zu finden und die Datei `index.php` soll im Unterordner `wet2vo-examples/vo12/slim-framework/helloworld/public` liegen, so wird dies wie folgt angegeben:

```
1 $app->setBasePath("/wet2vo-examples/vo12/slim-framework/helloworld/public");
```

Fehlerbehandlung aktivieren: Damit aussagekräftige Fehlermeldungen ausgegeben werden, wird die Error-Middleware registriert. In Slim 4 ist diese – anders als in früheren Versionen – nicht mehr standardmäßig aktiv; ohne sie führt eine geworfene Exception oder ein nicht gefundener Pfad zu einer (unschönen) PHP-Fehlermeldung.

```
1 $app->addErrorMiddleware(true, false, false);
```

Der erste Parameter aktiviert die Anzeige der Fehlerdetails, die beiden weiteren steuern das Logging der Fehler bzw. der Fehlerdetails. Die Anzeige der Details (`true`) ist ausschließlich für die Entwicklung vorgesehen; in Produktion ist dieser Wert auf `false` zu setzen, um keine internen Informationen (Pfade, Stacktraces) preiszugeben. Eine ausführlichere Betrachtung der Middleware folgt in Abschnitt 12.2.4.

Route definieren: Um etwas anzuzeigen, wird mindestens eine Route benötigt. In Slim stehen in der `\Slim\App`-Instanz dazu Methoden für jede HTTP-Methode zur Verfügung. Die wichtigste ist `get()`, die es ermöglicht, auf einen GET-Request unter einem bestimmten URL zu reagieren. In diesem Fall wird mit `/` spezifiziert, dass direkt das Verzeichnis, in dem die Datei `index.php` liegt, gemeint sein soll. Der zweite Parameter ist

die Callback-Methode, die aufgerufen wird, wenn ein GET-Request mit dem passenden URL eintrifft. Diese Methode enthält immer drei Parameter:

- **\$request**: Das Request-Objekt, welches alle Daten des aktuellen Requests enthält.
- **\$response**: Das Response-Objekt, welches an den Client zurückgegeben werden soll. Es kann (und soll) hier verändert werden.
- **\$args**: Ein Array mit benannten Platzhaltern der Route. Siehe dazu Abschnitt 12.2.5.

Um die „Hello world!“ Ausgabe zu erzeugen, wird das Response-Objekt benötigt. Über `getBody()` wird auf den Body, d. h. den Inhalt dieses Objekts zugegriffen und mittels `write()` der Text „Hello world!“ geschrieben. Anschließend wird das Response-Objekt zurückgegeben.

Applikation ausführen: Um die Applikation zu starten, muss schließlich die Methode `run()` des App-Objekts aufgerufen werden. Sie setzt das Routing (und andere Funktionalitäten) in Gang.

12.2.3 Dependencies

Um nun mit Slim praktisch arbeiten zu können, sollen zwei Dependencies (zusätzliche Softwarekomponenten) hinzugefügt werden. Zunächst sollen die Logging-Komponente *Monolog* verwendet werden, um einfach Debug-Ausgaben in eine Log-Datei (unabhängig von der des Webservers) schreiben zu können, weiters sollen *Latte*-Templates zum Einsatz kommen, um komplexere HTML-Ausgaben als im Hello-World-Beispiel machen zu können.

Dazu wird eine zweite Slim-Applikation namens `slimexample` angelegt. Zunächst wird der gleichnamige Ordner erstellt und Slim sowie die PSR-7-Komponente wie beim Hello World-Beispiel mit Composer installiert:

```
1 $ composer require slim/slim slim/psr7
```

In der Datei `public › index.php` wird nun – anders als beim Hello-World-Beispiel – zunächst ein Dependency-Container erstellt und konfiguriert, bevor das App-Objekt erzeugt wird. Der Grund dafür wird im nächsten Abschnitt deutlich: Der Container muss der `AppFactory` bekannt sein, *bevor* diese die Applikation erzeugt.

Dependency Injection Container

Um Dependencies einzubinden, benötigt Slim einen sogenannten Dependency Container. Einzubindende Komponenten werden zunächst in diesem Container registriert und konfiguriert. Wenn sie dann im Code benötigt werden, werden sie vom Container erzeugt und bereitgestellt.

Slim stellt selbst keinen Container bereit, kann aber mit jeder Komponente, die den PSR-11 Standard für Dependency Injection Container implementiert, arbeiten. Eine bekannte Dependency-Container-Komponente, die dieses Kriterium erfüllt, ist PHP-DI¹⁰. Diese wird zunächst mit Composer eingebunden:

```
1 $ composer require php-di/php-di
```

¹⁰<https://php-di.org/>

Container erzeugen: Um einen Dependency-Container zu erzeugen, existiert die Klasse `\DI\ContainerBuilder`. Sie erlaubt es, den Container zunächst zu konfigurieren und ihn dann mit einer Factory-Methode zu erzeugen. Anschließend wird der so erstellte Container der `AppFactory` mittels `setContainer()` übergeben. Dies muss vor dem Erzeugen des App-Objekts geschehen:

```
1 $containerBuilder = new ContainerBuilder();
2 // Definitionen werden hier hinzugefügt (siehe unten)
3 $container = $containerBuilder->build();
4 AppFactory::setContainer($container);
5
6 $app = AppFactory::create();
7 $app->setBasePath("/wet2vo-examples/vo12/slim-framework/slimexample/public");
```

Einstellungen hinterlegen: In diesen Container können nun verschiedene Dinge hinterlegt werden. Im einfachsten Fall sind dies simple Werte in Form eines assoziativen Arrays. Dies wird gerne verwendet, um Einstellungen der Applikation oder auch einzelner Dependencies zentral zu hinterlegen.

Dazu werden beim `ContainerBuilder`-Objekt sogenannte Definitionen (engl. „definitions“) in Form eines assoziativen Arrays hinzugefügt. Diese Definitionen erzeugen einen Eintrag im Container-Objekt unter einem frei wählbaren Namen. Dies kann ein beliebiger String sein, für die Einstellungen lautet der Eintrag dabei oft `settings`. Der Wert ist in der Regel ein assoziatives Array mit den verschiedenen Einstellungen.

```
1 $containerBuilder->addDefinitions(
2     [
3         "settings" => [
4             "settingName" => "settingValue",
5         ],
6     ],
7 );
```

Im Folgenden werden für die Applikation drei Einstellungen hinterlegt, die später für die Fehlerbehandlung in Abschnitt 12.2.4 herangezogen werden. Diese sind die drei Werte der Error-Middleware, die auch schon beim Hello World-Beispiel verwendet wurden: Ob Fehlermeldungen im Browser ausgegeben werden, ob Fehler (in eine Datei) geloggt werden sollen und ob Details ins Log geschrieben werden sollen. Alle drei Einstellungen werden zunächst als aktiv gesetzt.

```
1 $containerBuilder->addDefinitions(
2     [
3         "settings" => [
4             "displayErrorDetails" => true,
5             "logErrors" => true,
6             "logErrorDetails" => true,
7         ],
8     ],
9 );
```

Wichtig ist dabei, dass alle Einstellungen Teil *einer* einzigen `settings`-Definition sind. Die folgenden Ausschnitte zeigen jeweils nur die hinzukommenden Einträge innerhalb desselben Arrays. Ein erneuter Aufruf von `addDefinitions()` mit dem Schlüssel `settings` würde die zuvor gesetzten Werte überschreiben, da PHP-DI gleichnamige Definitionen nicht zusammenführt, sondern ersetzt.

Dependencies registrieren: Die Hauptaufgabe des Containers ist es jedoch, Dependencies als Service zu registrieren. Auch hier wird eine Definition hinzugefügt, als Name wird jedoch in der Regel der Klassenname der Dependency verwendet, da so automatisch erkannt werden kann, wenn eine Instanz benötigt wird. Unter diesem Namen (den man einfach durch das Anhängen der Klassenkonstante `::class` an den Klassennamen als String erhält) wird nun eine Funktion (genannt Closure) hinterlegt. In dieser wird die Dependency angelegt und zurückgegeben. Der Container ruft diese Funktion zur Erstellung der Dependency dann auf, wenn sie später das erste Mal benötigt wird.

Um auf den Container zugreifen zu können, um etwa Einstellungen zu entnehmen, wird dieser als Parameter mitgegeben. Über die Methode `get()` können die Werte unter dem hinterlegten Schlüssel entnommen werden.

```
1 $containerBuilder->addDefinitions(
2     [
3         MyService::class => function (ContainerInterface $container) {
4             $settings = $container->get("settings");
5             return new MyService($settings);
6         },
7     ],
8 );
```

Logger-Dependency: Monolog

Während der Entwicklung einer Applikation aber auch im Betrieb ist gutes Logging ein praktisches Tool, um Informationen über den Verlauf der Applikation zu erhalten. Nicht immer kann ein Debugger eingesetzt werden bzw. ist praktisch. *Monolog* ist das bekannteste Logging-Framework. Es basiert auf dem PSR-3 Logging Standard und lässt sich einfach über Composer installieren:

```
1 $ composer require monolog/monolog
```

Grundeinstellungen für den Logger (Name, Pfad der Log-Datei und Minimum-Log-Level) werden im `settings`-Eintrag des Containers unter dem Schlüssel `logger` hinterlegt:

```
1 $containerBuilder->addDefinitions(
2     [
3         "settings" => [
4             // Einstellungen zum Errorhandling
5             "logger" => [
6                 "name" => "slimexample-logger",
7                 "path" => "../logs/app.log",
8                 "level" => Level::Debug,
9             ],
10        ],
11    ],
12 );
```

Nun wird, wie soeben beschrieben, eine Definition mit dem Namen des LoggerInterfaces im Container angelegt. Hier wird bewusst nicht der Klassenname von Monolog sondern das generische PSR-3 Interface verwendet, um den Logger austauschbar zu machen. Im Code muss später nur angegeben werden, dass eine Instanz des LoggerInterfaces angelegt werden soll. Welche konkrete Implementierung dahinter liegt, spielt

keine Rolle. Dies ist wiederum lose Kopplung (engl. „loose coupling“), was ein erstrebenswertes Ziel in der Softwareentwicklung ist.

```

1 $containerBuilder->addDefinitions(
2   [
3     // Einstellungen wie zuvor beschrieben
4     LoggerInterface::class => function (ContainerInterface $container) {
5       $settings = $container->get("settings");
6       $loggerSettings = $settings["logger"];
7
8       $logger = new Logger($loggerSettings["name"]);
9
10      $fileHandler = new StreamHandler($loggerSettings["path"],
11      $loggerSettings["level"]);
12      $logger->pushHandler($fileHandler);
13
14      return $logger;
15    },
16  );

```

Zunächst werden aus dem Container die Einstellungen für den Logger abgefragt. Der Container wird dabei in der Closure als Parameter `$container` mitgegeben. Dann wird eine neue `\Monolog\Logger` Instanz angelegt – der Name wird aus den Einstellungen übernommen.

Um den Zielort bzw. die Art der Ausgabe zu definieren, kommt ein Objekt der Klasse `\Monolog\Handler\StreamHandler` zum Einsatz. Dies ermöglicht das Loggen in eine Datei (Dateien liegen sogenannten Streams – Datenströme – zugrunde). Als Argumente werden der Pfad und das Log-Level für jeden Eintrag mitgegeben. Im Anschluss wird dieser Handler der Logger-Instanz zugewiesen und das fertige Logger-Objekt zurückgegeben.

Sobald diese Zuweisung aktiv ist, kann über den Eintrag im Container mittels `get()` auf die Logging-Instanz zugegriffen werden. Dabei wird diese erst angelegt, wenn sie das erste Mal benötigt wird und belegt somit nicht unnötig Ressourcen.

```

1 $logger = $container->get(LoggerInterface::class);
2 $logger->info("Logger dependency created.");

```

Dies erzeugt (falls noch nicht vorhanden) eine Datei `logs › app.log` und erstellt folgenden Eintrag:

```
[2026-06-23T06:50:00+00:00] slimexample-logger.INFO: Logger dependency created. [] []
```

Derartige Log-Aufrufe können an allen wichtigen Stellen eingebaut werden und somit bei Problemen schnell Aufschluss über die Ursache geben. Monolog bietet noch eine Vielzahl an weiteren Funktionalitäten. Diese können aus der Dokumentation¹¹ entnommen werden.

In Abschnitt 12.2.4 wird dieser Logger ebenfalls eingesetzt, um Fehlermeldungen in die Logdatei zu schreiben.

¹¹<https://github.com/Seldaek/monolog/blob/main/doc/01-usage.md>

Renderer: Latte

Wird das Slim Framework nicht für eine REST-API die nur Daten retour liefert, sondern zum Erstellen einer kompletten Webapplikation verwendet, ist die Verwendung eines Renderers, d. h. eines Template-Systems, unumgänglich. Dieser Renderer ermöglicht die Ausgabe komplexer HTML-Dokumente, die allerdings nicht direkt ausgegeben, sondern wieder in ein Response-Objekt verpackt werden müssen.

Slim selbst bietet mit `slim/php-view` eine einfache Komponente zur Ausgabe mit reinem PHP an: HTML-Seiten werden mit PHP-Echo-Tags erstellt, und Slim übergibt diesen Seiten die Daten, die dann ausgegeben werden. Für komplexere Anwendungen empfiehlt sich jedoch eine vollwertige Template-Engine. In dieser Vorlesung kommt wie immer *Latte*¹² zum Einsatz. Die Integration in Slim folgt der offiziellen Anleitung von Daniel Opitz¹³.

Zunächst wird Latte mittels Composer installiert:

```
1 $ composer require latte/latte
```

Dann werden im Container für das Template-System entsprechende Einstellungen hinterlegt:

```
1 $containerBuilder->addDefinitions(
2     [
3         "settings" => [
4             // Einstellungen für Errorhandling und Logger
5             "template" => [
6                 "path" => "../templates",
7                 "cache" => "../cache",
8                 "auto_refresh" => true,
9             ],
10    ],
11 ],
12 );
```

Unter dem Schlüssel `template` werden der Pfad zu den Templates, das Cache-Verzeichnis für die kompilierten Templates und der Status des Auto-Refresh angegeben.

Damit aus den Templates heraus URLs anhand von Routennamen erzeugt werden können (wie weiter unten bei der Formularverarbeitung gezeigt), benötigt die Latte-Engine den *RouteParser* von Slim. Dieser übersetzt einen Routennamen in den zugehörigen Pfad und stellt dabei automatisch den Base-Path der Applikation voran. Der *RouteParser* stammt aus dem App-Objekt und wird – nach dessen Erstellung – im Container hinterlegt, sodass die Engine-Definition später darauf zugreifen kann:

```
1 $container->set("routeParser", $app->getRouteCollector()->getRouteParser());
```

Im Anschluss wird die Latte-Engine im Container als Service registriert. In der Definition wird zusätzlich über `addFunction()` eine eigene Latte-Funktion `url_for` hinterlegt, die einen Routennamen entgegennimmt und daraus den vollständigen Pfad (inklusive Base-Path) erzeugt:

```
1 $containerBuilder->addDefinitions(
2     [
3         // Einstellungen und Logger-Dependency
```

¹²<https://latte.nette.org/>

¹³<https://latte.nette.org/en/cookbook/slim-framework>

```

4     Engine::class => function (ContainerInterface $container) {
5         $settings = $container->get("settings");
6         $templateSettings = $settings["template"];
7
8         $latte = new Engine();
9         $latte->setLoader(new FileLoader($templateSettings["path"]));
10        $latte->setCacheDirectory($templateSettings["cache"]);
11        $latte->setAutoRefresh($templateSettings["auto_refresh"]);
12
13        $routeParser = $container->get("routeParser");
14        $latte->addFunction("url_for",
15            fn(string $name, array $data = [], array $query = []): string
16                => $routeParser->urlFor($name, $data, $query),
17        );
18
19        return $latte;
20    },
21 ],
22 );

```

Aus dem Container werden zunächst die Einstellungen abgerufen. Mit ihnen wird eine `\Latte\Engine`-Instanz konfiguriert: Der `FileLoader` legt das Template-Verzeichnis fest, `setCacheDirectory()` das Verzeichnis für die kompilierten (und damit schnellen) Templates und `setAutoRefresh()` steuert, ob Latte den Cache bei Änderungen automatisch neu erzeugt – im Produktivbetrieb wird dies aus Performance-Gründen deaktiviert.

Abschließend wird über `addFunction()` die Funktion `url_for` in der Engine registriert. Sie steht damit in allen Templates (wie beim *foo/router-skeleton*) zur Verfügung und übersetzt einen Routennamen mithilfe des `RouteParsers` in den zugehörigen Pfad. Auf diese Weise bleibt die URL-Erzeugung an einer einzigen Stelle gebündelt und muss nicht in jedem Routing-Callback wiederholt werden. Da die Engine-Definition eine Closure ist, die erst beim ersten Rendern ausgewertet wird, sind die zuvor mit `$container->set()` hinterlegten Werte zu diesem Zeitpunkt bereits vorhanden.

Die Engine-Instanz allein genügt jedoch noch nicht, um direkt ein PSR-7-Response-Objekt zu erzeugen. Gemäß der offiziellen Anleitung wird daher eine schlanke Hilfsklasse `TemplateRenderer` erstellt, die das gerenderte Template in das Response-Objekt schreibt. Sie wird in `src▶Renderer▶TemplateRenderer.php` abgelegt:

```

1  <?php
2
3  namespace App\Renderer;
4
5  use Latte\Engine;
6  use Psr\Http\Message\ResponseInterface;
7
8  final class TemplateRenderer
9  {
10     public function __construct(
11         private Engine $engine,
12     ) {}
13
14     public function template(ResponseInterface $response, string $template, array
15         $data = []): ResponseInterface {
16         $output = $this->engine->renderToString($template, $data);

```

```
16     $response->getBody()->write($output);
17
18     return $response;
19 }
20 }
```

Die Klasse erhält im Konstruktor die `Engine`-Instanz. Da diese im Container registriert ist, erzeugt PHP-DI die `TemplateRenderer`-Instanz bei Bedarf automatisch (Autowiring) – eine eigene Definition oder eine zusätzliche Middleware ist nicht nötig. Damit die Klasse über Composer gefunden wird, ist das `src`-Verzeichnis per PSR-4 in der `composer.json` einzutragen:

```
1 "autoload": {
2   "psr-4": {
3     "App\\": "src/"
4   }
5 }
```

Nach dem Eintragen wird der Autoloader mit `composer dump-autoload` aktualisiert.

Um ein Template zu rendern, wird die `TemplateRenderer`-Instanz aus dem Container angefordert und ihre Methode `template()` aufgerufen. Diese rendert das Latte-Template und schreibt das Ergebnis in das Response-Objekt:

```
1 return $this->get(TemplateRenderer::class)->template($response, "template.latte",
    ["param1" => "value1"]);
```

Innerhalb eines Routing-Callbacks verweist `$this` dabei auf den Container. Die Anwendung dieses Aufrufs wird in Abschnitt 12.2.5 im Rahmen verschiedener Routen demonstriert. Latte escaped übergebene Werte standardmäßig kontextsensitiv, was einen wirksamen Schutz vor Cross-Site Scripting (XSS) darstellt.

Die Beispiele setzen voraus, dass am Beginn von `index.php` die jeweiligen Namespaces `\Latte\Engine`, `\Latte\Loaders\FileLoader`, `\Slim\Routing\RouteContext` und `\App\Renderer\TemplateRenderer` per `use` importiert wurden.

12.2.4 Middleware

Slim unterstützt sogenannte Middleware. Dies ist Code, der sowohl vor als auch nach dem Start der Slim-App ausgeführt wird und dabei die Request- und Response-Objekte auslesen oder verändern kann. Somit könnte z. B. ein automatischer Schutz vor Cross-Site Request Forgery (CSRF) bei jedem Request implementiert oder jeder Request automatisch authentifiziert werden. Auch Caching ist möglich.

Middleware in Slim funktioniert dabei nach dem Zwiebel-Prinzip. Hinzugefügte Middleware wird dabei wie Ringe um die eigentliche Applikationslogik gelegt. Beim Request werden zuerst die Middlewareschichten durchlaufen, dann wird die Applikationslogik zur Generierung der Response ausgeführt und danach wird die Middleware wieder in umgekehrter Reihenfolge ausgeführt. Abbildung 12.3 stellt dies dar.

Technisch basiert Middleware in Slim auf dem Standard PSR-15¹⁴. Eine Middleware kann entweder als Closure umgesetzt werden oder als Klasse, die das Interface

¹⁴<https://www.php-fig.org/psr/psr-15/>

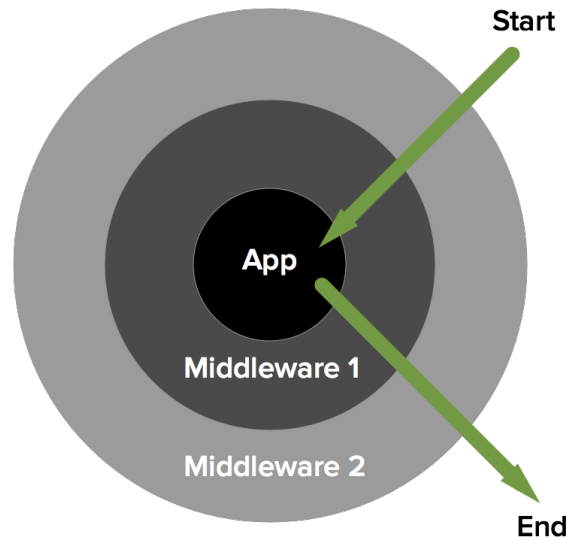


Abbildung 12.3: Middleware wird vor der Applikationslogik von außen nach innen (LIFO – last in, first out) und nach der Applikationslogik erneut von innen nach außen ausgeführt. Quelle: [5].

`Psr\Http\Server\MiddlewareInterface` implementiert. Dieses verlangt eine Methode `process()`, die das Request-Objekt sowie einen Request-Handler erhält. Innerhalb dieser Methode kann der Request verändert, mit `$handler->handle($request)` an den nächsten Handler weitergereicht und die zurückkommende Response anschließend nachbearbeitet werden.

Eine Middleware, die Caching übernimmt, analysiert den ankommenden Request. Ist eine gecachte Antwort vorhanden, wird diese sofort zurückgegeben und die Applikation wird nicht mehr betreten. Gab es keine gecachte Version, wird die Response in der Applikation erstellt und danach speichert die Middleware die Response nach dem Verlassen der App im Cache, sodass sie beim nächsten Aufruf zur Verfügung steht.

Ein weiteres Einsatzgebiet ist Error-Handling: um PHP Exceptions abzufangen, kann eine mit Slim mitgelieferte Error-Handling-Middleware registriert werden.

Middleware wird im Gegensatz zu Dependencies zum fertigen App-Objekt hinzugefügt. Für das Error-Handling geschieht dies mittels `addErrorMiddleware()`. Diese Methode hat drei verpflichtende und einen optionalen booleschen Parameter:

- `$displayErrorDetails`: Zeigt Fehlermeldungen an. `true` während der Entwicklung, `false` im Produktivbetrieb.
- `$logErrors`: Aktiviert das Logging von Fehlern.
- `$logErrorDetails`: Zeigt Details im Fehlerlog an.
- `$logger`: Optional kann im vierten Parameter eine Instanz eines PSR-3 kompatiblen Loggers angegeben werden.

Die Werte der ersten drei Einstellungen wurden bereits im Container unter `settings` (jeweils mit dem Wert `true`) hinterlegt, können also von dort abgefragt werden. Als Logger wird das Monolog-Objekt verwendet, das ebenfalls schon über den Container angefordert und erzeugt wurde.

```
1 $settings = $container->get("settings");
2 $app->addErrorMiddleware(
3     $settings["displayErrorDetails"],
4     $settings["logErrors"],
5     $settings["logErrorDetails"],
6     $logger,
7 );
8 $logger->info("Error middleware added.");
```

Tritt nun ein Fehler auf, so wird dieser am Bildschirm angezeigt und ebenfalls mit Details in die Logdatei geschrieben.

Weitere Einstellungen für die Error-Middleware sind in der Slim-Dokumentation¹⁵ einsehbar. Darüber hinaus gibt es noch weitere Middleware, die mit Slim mitgeliefert wird, wie das Routing, Output-Buffering oder Bestimmung der Content-Length.

12.2.5 Routing

Das Herzstück jeder Slim-Applikation sind ihre Routen. Diese bestimmen, welche Pfade in der Applikation aufgerufen werden können und entsprechende Ergebnisse liefern. Eine Route besteht aus einem Protokoll (GET, POST, PUT, DELETE, OPTIONS, PATCH), einem URI (d. h. URL) und einer Callback-Methode, die aufgerufen wird, wenn ein Request auf diese Route eingeht. Die HTTP-Methode wird über eine von sechs Methoden des App-Objekts ausgewählt, URI und Callback sind deren Parameter. Für einen GET-Request sieht dies für das Hello-World-Beispiel wie folgt aus:

```
1 $app->get("/", function (Request $request, Response $response, array $args) {
2     $response->getBody()->write("Hello World!");
3     return $response;
4 });
```

Die Methode heißt `get()` und reagiert damit nur auf eingehende GET-Requests. / legt fest, dass das Root-Verzeichnis, in dem sich die aufrufende Datei befindet, gemeint ist. Angenommen, der Code liegt im Docker-Container in der Datei `index.php` im Ordner `var/www/html/wet2vo-examples/vo12/slim-framework/slimexample/public`, dann reagiert diese Route auf `http://localhost:8080/wet2vo-examples/vo12/slim-framework/slimexample/public/`. Voraussetzung ist ein korrekt gesetzter Base-Path.

Der Callback hat immer drei Argumente: das Request-Objekt, aus dem alle Details des Requests ausgelesen werden können, das Response-Objekt, in das die Response geschrieben werden muss und benannte Argumente, falls das Latte-Template dieser Route Platzhalter enthält. Die Request- und Response Objekte sollten, wie schon gezeigt, nicht die konkreten Implementierungen (etwa von `\Slim\Psr7`) verwenden, sondern nur auf das jeweilige PSR-Interface (`Psr\Http\Message\ResponseInterface` bzw. `Psr\Http\Message\ServerRequestInterface`) verwenden. Somit ist es jederzeit möglich, die PSR-7 Implementierung zu tauschen (lose Kopplung).

Im Callback selbst kann mittels `getBody()` der Inhalt der Response abgefragt und mittels `write()` beschrieben werden. Auch mittels `echo` ausgegebene Inhalte werden standardmäßig an die Response gehängt. Dann wird das Response-Objekt zurückgegeben und von Slim angezeigt.

¹⁵<https://www.slimframework.com/docs/v4/middleware/error-handling.html>

Durch das Hinzufügen der Slim Routing Middleware kann beim Routing Einfluss auf die Reihenfolge der Middleware genommen werden. Da sie für einige andere Middlewares verwendet wird, empfiehlt sich das Hinzufügen:

```
1 $app->addRoutingMiddleware();
```

Routing für eine Formularverarbeitung

Nachfolgend soll ein simples Routing für Formularverarbeitung erstellt werden. Dazu wird eine GET-Route benötigt, die das Formular anzeigt und eine POST-Route, die das Ergebnis des mit POST versendeten Formulars darstellt. Dargestellt wird alles mittels Latte-Templates, die wie in Abschnitt 12.2.3 gezeigt initialisiert wurden.

Zunächst wird eine GET-Route auf das Root-Verzeichnis definiert. Wird sie aufgerufen, soll das Formular – gespeichert in `templates/form.latte` – angezeigt werden. Auf den Container kann im Routing-Callback direkt über `$this->get()` zugegriffen werden, somit kann der `TemplateRenderer` angefordert und das Template gerendert werden. Es werden dabei keine Daten übergeben: Das Formularfeld bleibt leer, weil das Template über `{parameters}` einen Default für den Platzhalter deklariert (siehe unten), und den Pfad für das `action`-Attribut erzeugt das Template selbst über die Funktion `url_for`.

```
1 $app->get("/", function (Request $request, Response $response, array $args) {
2     return $this->get(TemplateRenderer::class)->template($response, "form.latte");
3 });
```

Das Template sieht wie folgt aus, die Ausgabe im Browser ist in Abbildung 12.4 ersichtlich:

```
1 {parameters string $placeholder = ''}
2 <!DOCTYPE html>
3 <html lang="en">
4     <head>
5         <meta charset="UTF-8">
6         <title>Tell Me Your Name</title>
7     </head>
8     <body>
9         <h1>Tell Me Your Name</h1>
10        <form action="{url_for('result')}" method="post">
11            <label for="name">Name:</label>
12            <input type="text" id="name" name="name" value="{ $placeholder }">
13            <button type="submit">Send</button>
14        </form>
15    </body>
16 </html>
```

Hinweis zum Template: Der `parameters`-Tag am Beginn deklariert die vom Template erwartete Eingabevariable `$placeholder` samt Typ und Default-Wert. Dadurch muss die GET-Route auf / keinen Platzhalter übergeben – fehlt er, greift der Default (leerer String). Ohne diese Deklaration würde der Zugriff auf eine nicht übergebene Variable eine PHP-Warnung auslösen.

Im Template erzeugt die Funktion `{url_for('result')}` den Pfad zur Route mit dem Namen `result`. Diese Funktion wurde – wie in Abschnitt 12.2.3 gezeigt – in der Latte-Engine registriert und greift intern auf den `RouteParser` zu. Der Vorteil: Ändert

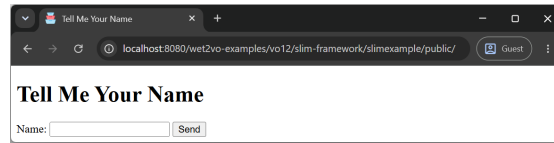


Abbildung 12.4: Die Slim-Applikation zeigt beim Aufruf des Root-Verzeichnisses das Formular an.

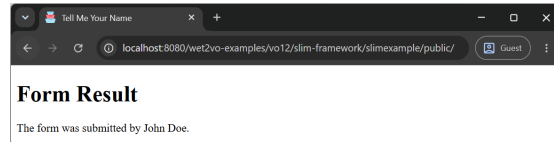


Abbildung 12.5: Das Ergebnis der Formularverarbeitung zeigt den eingegebenen Namen (John Doe) an.

sich der konkrete Pfad der Route, muss weder das Template noch der Handler angepasst werden. Die benannte Route `result` ist die POST-Route, die das Ergebnis verarbeitet:

```
1 $app->post("/", function (Request $request, Response $response, array $args) {
2     $data = $request->getParsedBody();
3     $name = $data["name"];
4
5     return $this->get(TemplateRenderer::class)->template($response, "result.latte",
6         ["name" => $name]);
7 }->setName("result");
```

Die Route zeigt auch auf das Root-Verzeichnis, wird jedoch nur aufgerufen, wenn mittels POST übertragen wird (was im `method`-Attribut des Formulars festgelegt ist). Mit Hilfe von `getParsedBody()` wird zunächst der gesamte Body als Array in `$data` gespeichert. Anschließend wird Parameter `name` über `$data["name"]` ausgelesen, dies entspricht einem Zugriff auf `$_POST["name"]`. Dieser Wert wird als Parameter ans Template `result.latte` übergeben, wo er angezeigt wird. Um die Route wie oben gezeigt über `url_for` referenzieren zu können, wird ihr mit `setName()` ein Name zugewiesen. Dies hat den Vorteil, dass die Route geändert werden kann (sie könnte z. B. auch `/result` lauten) und das Template enthält automatisch den korrekten Pfad.

Das Resultat-Template ist wie folgt aufgebaut und rendert wie in Abbildung 12.5 dargestellt.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>Tell Me Your Name</title>
6   </head>
7   <body>
8     <h1>Form Result</h1>
9     <p>The form was submitted by {$name}.</p>
10  </body>
11 </html>
```



Abbildung 12.6: Wird die Seite mit „John%20Doe“ nach dem Root-Slash aufgerufen, wird dieser Name ausgelesen und an das Template übergeben. Dort wird es als **value** des Eingabefeldes gesetzt, sodass der Name dort aufscheint.

Platzhalter in der Route

Routen müssen nicht auf fixe URL-Pfade lauten, stattdessen können auch Platzhalter angegeben werden, die beliebige Werte zulassen. Diese Werte werden im Callback im dritten Parameter **\$args** mitübergeben. Um einen Platzhalter zu definieren, wird dieser unter Angabe eines Namens in geschwungene Klammern gesetzt. Im Array **\$args** ist er dann unter diesem Namen verfügbar.

Im folgenden Beispiel wird für das Formular noch eine zweite GET-Route definiert, die es erlaubt, die URL mit einem beliebigen Platzhalter aufzurufen. Dieser wird ausgelesen und ins Template übergeben, wo er ins **value**-Attribut für das Formularfeld gesetzt wird. Somit ist es möglich, durch Aufrufen eines eigenen URLs das Formularfeld quasi vor auszufüllen. Die Route dazu sieht wie folgt aus:

```
1 $app->get(
2    ("/{placeholder}[/]",
3     function (Request $request, Response $response, array $args) {
4         return $this->get(TemplateRenderer::class)->template($response, "form.latte",
5             ["placeholder" => $args["placeholder"]]);
6     });
```

Als Schlüssel dient der Name **placeholder**. Sein Wert kann beliebig sein und wird dem Template als Parameter **placeholder** übergeben. Der abschließende Schrägstrich ist durch Angabe in eckigen Klammern optional, d. h. die Route funktioniert sowohl mit als auch ohne. Abbildung 12.6 stellt das Ergebnis des Formulars dar, wenn die Seite mit <https://localhost:8080/wet2vo-examples/vo12/slim-framework/slimexample/public/John%20Doe> aufgerufen wurde.

Die als Platzhalter erlaubten Zeichen können hierbei etwa auch mit Regular Expressions eingeschränkt werden. Details finden sich in der Dokumentation¹⁶.

Auftrennen der einzelnen Schritte im Slim-Skeleton

Vorerst wurden die Einstellungen (settings), das Konfigurieren des Containers (dependencies injection) und das Erstellen der Routen (routing) in der Datei **index.php** durchgeführt, das als Startpunkt für die Webseite fungiert.

Bei der Verwendung des Slim-Skeletons sind diese Schritte in einzelne PHP-Dateien in einem Unterverzeichnis **app** aufgeteilt und nur noch über **require** in die **index.php** eingebunden.

Die Datei **index.php** im Verzeichnis **public** ändert sich dadurch für alle auf dem Slim-Skeleton basierenden Projekte auf wie folgt (Anmerkung: der Slim-Skeleton enthält

¹⁶<https://www.slimframework.com/docs/v4/objects/routing.html#route-placeholders>

noch mehr Beispielcode, daher sind noch weitere Dinge in der Hauptdatei eingebunden, die aber vorerst nicht weiter wichtig sind):

```
1 <?php
2 declare(strict_types=1);
3
4 // use-Statements
5
6 require __DIR__ . '/../vendor/autoload.php';
7
8 // Instantiate PHP-DI ContainerBuilder
9 $containerBuilder = new ContainerBuilder();
10
11 if (false) { // Should be set to true in production
12     $containerBuilder->enableCompilation(__DIR__ . '/../var/cache');
13 }
14
15 // Set up settings
16 $settings = require __DIR__ . '/../app/settings.php';
17 $settings($containerBuilder);
18
19 // Set up dependencies
20 $dependencies = require __DIR__ . '/../app/dependencies.php';
21 $dependencies($containerBuilder);
22
23 // Set up repositories
24 $repositories = require __DIR__ . '/../app/repositories.php';
25 $repositories($containerBuilder);
26
27 // Build PHP-DI Container instance
28 $container = $containerBuilder->build();
29
30 // Instantiate the app
31 AppFactory::setContainer($container);
32 $app = AppFactory::create();
33 $callableResolver = $app->getCallableResolver();
34
35 // Register middleware
36 $middleware = require __DIR__ . '/../app/middleware.php';
37 $middleware($app);
38
39 // Register routes
40 $routes = require __DIR__ . '/../app/routes.php';
41 $routes($app);
42
43 /** @var SettingsInterface $settings */
44 $settings = $container->get(SettingsInterface::class);
45
46 $displayErrorDetails = $settings->get('displayErrorDetails');
47 $logError = $settings->get('logError');
48 $logErrorDetails = $settings->get('logErrorDetails');
49
50 // Create Request object from globals
51 $serverRequestCreator = ServerRequestCreatorFactory::create();
52 $request = $serverRequestCreator->createServerRequestFromGlobals();
53
54 // Create Error Handler
```

```
55 $responseFactory = $app->getResponseFactory();
56 $errorHandler = new ErrorHandler($callableResolver, $responseFactory);
57
58 // Create Shutdown Handler
59 $shutdownHandler = new ShutdownHandler($request, $errorHandler, $displayErrorDetails
    );
60 register_shutdown_function($shutdownHandler);
61
62 // Add Routing Middleware
63 $app->addRoutingMiddleware();
64
65 // Add Body Parsing Middleware
66 $app->addBodyParsingMiddleware();
67
68 // Add Error Middleware
69 $errorMiddleware = $app->addErrorMiddleware($displayErrorDetails, $logError,
    $logErrorDetails);
70 $errorMiddleware->setDefaultErrorHandler($errorHandler);
71
72 // Run App & Emit Response
73 $response = $app->handle($request);
74 $responseEmitter = new ResponseEmitter();
75 $responseEmitter->emit($response);
```

12.3 Einführung in Testing

Das Testen von Applikationen gehört zu einer der wichtigsten Aufgaben im Softwareentwicklungsprozess. Ziel ist es, sicherzustellen, dass Webanwendungen funktionieren und dieser Zustand auch dann erhalten bleibt, wenn sie kontinuierlich weiterentwickelt werden. Vor allem, wenn Software veröffentlicht wird, sodass sie von anderen Personen verwendet wird, sollte mehr getan werden als nur zu hoffen, dass alles so funktioniert, wie es zum Zeitpunkt der Entwicklung angedacht war. Software Tests schaffen genau in diesem Punkt Sicherheit.

Tests werden jedoch allzu oft aufgrund von mangelnder Zeit außen vor gelassen. Dieses Argument ist nicht völlig von der Hand zu weisen, denn das Aufsetzen einer Testing-Infrastruktur sowie das Schreiben von sinnvollen Test-Szenarien benötigt in der Tat Zeit. Dies ist in der Regel jedoch gut investiert, da sie später durch das Vermeiden von Fehlern wieder gewonnen wird. Meist wird auch mehr Zeit eingespart, als ohne Tests.

12.3.1 Zeitpunkt für Testing

Das Schreiben von Tests wird oft ans Ende des Entwicklungsprozesses verlagert. Nachdem eine Webapplikation läuft, werden Testszenarien erstellt, um diese zu überprüfen. Eigentlich sollte Testen aber eine Aktivität sein, die bereits vor der eigentlichen Entwicklung begonnen wird. Zunächst werden die Testing Tools (siehe Abschnitt 12.4) installiert und konfiguriert, sodass sie zu Beginn der Entwicklung bereit sind. Während des Entwicklungsprozesses werden dann zeitgleich mit neuen erstellten Klassen auch sofort Test-Klassen geschrieben, die die Funktionalität der neuen Klasse überprüfen.

Selbst wenn die Applikation abgeschlossen ist, werden oft neue Testszenarien noch hinzugefügt, da zum Entwicklungszeitpunkt kaum alle Fälle abgeschätzt werden können.

12.3.2 Was wird wie getestet?

Eine moderne PHP-Applikation besteht aus Klassen, Methoden und auch Funktionen. Getestet werden in der Regel alle öffentlichen Klassen, Methoden und Funktionen, d. h. solche, die mit dem Schlüsselwort `public` versehen sind und somit für den Aufruf durch Dritte erdacht wurden. Dieses Testen von einzelnen Teilen einer Applikation wird *Unit Testing* genannt, als Unit wird ein Teilstück der Applikation (eben eine Klasse, Methode oder Funktion) angesehen. Funktioniert dieses Teilstück einzeln für sich korrekt, kann auch davon ausgegangen werden, dass es im Zusammenspiel mit anderen Teilen in der Applikation funktioniert.

Da dies jedoch nicht immer der Fall ist (eine Applikation ist mehr als die Summe ihrer Teile), werden auch sogenannte *funktionale Tests* durchgeführt, die das Verhalten der Applikation im Gesamten überprüfen.

Moderne Ansätze der Softwareentwicklung rücken Testing stark in den Fokus. Bei *Test Driven Development* (TDD) etwa werden Tests vor dem eigentlichen Applikations-Code erstellt. Die Tests schlagen dabei zunächst fehl und geben somit Auskunft, was von der Applikation erwartet wird. Im Laufe der Implementierung ist es das Ziel, dass die Tests positiv durchgeführt werden und die Applikation die Testszenarien somit erfüllt. Ein weiterer Ansatz ist *Behavior-Driven Development* (BDD). In diesem Ansatz wird das Verhalten der Applikation in sogenannten Stories beschrieben, was einen mehr menschenfreundlicheren Testansatz darstellt. Mehr Informationen dazu finden sich etwa in [9].

12.4 PHPUnit

PHPUnit ist das gängigste Tool, um sogenannte Unit Tests, also das Testen einzelner Teile einer Webapplikation, durchzuführen. Tests, die mit PHPUnit geschrieben werden, werden *Test Cases* genannt, welche wiederum in *Test Suites* zusammengefasst werden. Der *Test Runner* führt eine Test Suite dann aus.

Ein Test Case ist eine Klasse (vom Typ `\PHPUnit\Framework\TestCase`), welche Methoden enthält, die alle mit `test` beginnen und einzelne Aspekte einer Webapplikation testen. Jede dieser Methoden überprüft bestimmte Annahmen, diese können `true` oder `false` sein, wobei natürlich `true` angestrebt wird, da in diesem Fall alle Tests positiv abgeschlossen werden.

Eine Test Suite fasst mehrere Test Cases zusammen. Kleine Applikationen oder Komponenten kommen in der Regel mit einer Test Suite aus, große Applikationen können die Tests mit mehreren Suites aufteilen.

Der Test Runner ist die Art und Weise, wie Tests gestartet werden. Dies geschieht üblicherweise über die Commandline.

12.4.1 PHPUnit vorbereiten

Tests werden üblicherweise in einem Unterverzeichnis `tests` abgelegt. Dieses liegt auf der obersten Ebene der Applikation, genau wie `src`, `vendor` oder `public`.

PHPUnit selbst wird mit Composer installiert. Da es sich jedoch nicht um eine Komponente handelt, die zum Betrieb der eigenen Applikation nötig ist, sondern nur während der Entwicklung gebraucht wird, wird PHPUnit als sogenannte Development-Dependency mit dem `-dev` Flag installiert:

```
1 $ composer require --dev phpunit/phpunit
```

Composer installiert dabei die aktuelle Version (PHPUnit 13¹⁷), die PHP 8.4 oder neuer voraussetzt.

Um die Test Suites zu konfigurieren, wird die Datei `phpunit.xml` im Root des Projektes angelegt. Sie sieht wie folgt aus:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:noNamespaceSchemaLocation="https://schema.phpunit.de/13.2/phpunit.xsd"
4     bootstrap="tests/bootstrap.php">
5     <testsuites>
6         <testsuite name="favoriteSubject">
7             <directory>tests</directory>
8         </testsuite>
9     </testsuites>
10    <source>
11        <include>
12            <directory>src</directory>
13        </include>
14    </source>
15 </phpunit>
```

Darin wird festgelegt, dass es im Verzeichnis `tests` eine Datei `bootstrap.php` liegt, die bei den Tests ausgeführt werden soll. Darin wird der Composer Autoloader aufgerufen, damit auch für die Tests alle Komponenten vorhanden sind.

Im Anschluss wird eine Test Suite (mit `<testsuite>`) definiert, welche die auszuführenden Tests festlegt. Diese werden im Verzeichnis `tests` angelegt. PHPUnit berücksichtigt dabei standardmäßig alle Dateien, die mit `Test.php` enden.

Um festzulegen, welche Quellcode-Dateien berücksichtigt werden sollen (etwa für eine Code-Coverage), wird dies mit dem `<source>`-Element angegeben, um alle Dateien mit der Endung `.php` im Ordner `src` zu berücksichtigen.

Die Datei `tests/bootstrap.php` besteht nur aus einer Codezeile und bindet den Composer Autoloader ein:

```
1 require dirname(__DIR__) . "/vendor/autoload.php";
```

12.4.2 Die Code-Klasse FavoriteMTDSubject

Getestet werden soll eine Klasse, die das Lieblingsfach in MTD speichert und ausgeben kann und mit einer weiteren Methode auf eine ähnliche Aussage reagieren kann. Sie sieht wie folgt aus:

¹⁷<https://phpunit.de/>

```

1 <?php
2
3 namespace PHPUnitExample;
4
5 use Exception;
6 use InvalidArgumentException;
7
8 class FavoriteMTDSubject
9 {
10     private(set) string $favoriteSubject;
11
12     public function __construct(string $favoriteSubject)
13     {
14         if (mb_trim($favoriteSubject) === "") {
15             throw new InvalidArgumentException("Favorite subject cannot be empty. You must
16             choose!");
17         }
18         $this->favoriteSubject = $favoriteSubject;
19     }
20     public function say(): string
21     {
22         return "The best subject in MTD is $this->favoriteSubject!";
23     }
24
25     public function respondTo(string $input): string
26     {
27         $input = mb_strtolower($input);
28         $myFavoriteSubject = mb_strtolower($this->favoriteSubject);
29
30         if (mb_strpos($input, $myFavoriteSubject) === false) {
31             throw new Exception("Never! $this->favoriteSubject is the best subject in MTD!
32             ");
33         }
34         return "Absolutely true!";
35     }
36 }

```

Im Konstruktor kann als Argument das Lieblingsfach (in der Regel Web Technologies) gesetzt werden. Wird als Argument eine leere Zeichenkette übergeben, wird eine `InvalidArgumentException` geworfen. Die Methode `say()` gibt einen String mit dem Lieblingsfach retour und mittels `respondTo()` kann auf eine andere Ansage zum Thema Lieblingsfach reagiert werden. Stimmt das Fach überein, wird zugestimmt, stimmt es nicht überein, wird eine Exception geworfen.

12.4.3 Die Test-Klasse FavoriteMTDSubjectTest

In der Testklasse werden nun vier Tests Cases erstellt, die das Verhalten der öffentlichen Methoden der Code-Klasse überprüfen sollen. Getestet werden das Verhalten des Konstruktors und der `say()`-Methode sowie die zwei möglichen Ergebnisse der `respondTo()`-Methode.

Test 1: Konstruktor

Für den Konstruktor muss getestet werden, dass das übergebene Argument kein leerer String ist. In diesem Fall wird eine `InvalidArgumentException` geworfen, also sieht auch der Unit-Test so aus, dass er dies als erwartetes Verhalten ansieht. Mittels der `expectException()`-Methode von PHPUnit wird signalisiert, dass in den darauffolgenden Zeilen eine `InvalidArgumentException` erwartet wird. Danach wird ein neues Objekt der `FavoriteMTDSubject`-Klasse mit einem leeren String als Parameter angelegt. Wird dabei eine Exception ausgelöst, so gilt der Test als bestanden.

```
1 public function testConstructorThrowsExceptionOnEmptyFavoriteSubject()
2 {
3     $this->expectException(InvalidArgumentException::class);
4     new FavoriteMTDSubject("");
5 }
```

Test 2: say()

Für die Methode `say()` muss getestet werden, ob der String, den diese zurückgibt, den Namen des Lieblingsfachs enthält. Dies kann mit `assertEquals()` erfolgen, das zwei Werte vergleicht:

```
1 public function testSayFavoriteSubject()
2 {
3     $subject = new FavoriteMTDSubject("Web Technologies");
4     $this->assertEquals("The best subject in MTD is Web Technologies!", $subject->say
5         ());
6 }
```

Der erste Wert ist der erwartete String, der zweite ist der tatsächliche Aufruf der Methode am zuvor angelegten Objekt.

Test 3: respondTo() bei Übereinstimmung

Um zu testen, ob `respondTo()` korrekt funktioniert, wird getestet, ob der String „Absolutely true!“ zurückgegeben wird, wenn der Name des hinterlegten Lieblingsfachs mit dem aus einer anderen Aussage übereinstimmt. Wiederum kommt `assertEquals()` zum Einsatz.

```
1 public function testRespondToInAgreement()
2 {
3     $subject = new FavoriteMTDSubject("Web Technologies");
4     $opinion = "Web Technologies is the best!";
5
6     $this->assertEquals("Absolutely true!", $subject->respondTo($opinion));
7 }
```

Zunächst wird ein Objekt angelegt und dann eine weitere Aussage. Diese muss nicht unbedingt komplett identisch mit der, die von `say()` generiert wird sein, da `respondTo()` nur überprüft, ob das Lieblingsfach enthalten ist (nicht, ob der Satz identisch ist). Dann wird `respondTo()` aufgerufen und mit dem erwarteten Rückgabestring verglichen.

Test 4: `respondTo()` bei Uneinigkeit

Schließlich wird noch überprüft, wenn keine Einigkeit über das Lieblingsfach in MTD herrscht. In diesem Fall wirft `respondTo()` eine Exception. Dies wird mit der Methode `expectException()` überprüft. Übergeben wird der Name der erwarteten Exception. Wird eine Exception geworfen, gilt der Test als bestanden.

```
1 public function testRespondToInDisagreement()
2 {
3     $subject = new FavoriteMTDSUBJECT("Web Technologies");
4     $opinion = "I love Algorithmic Thinking!";
5
6     $this->expectException(Exception::class);
7
8     $subject->respondTo($opinion);
9 }
```

12.4.4 Tests aufrufen

Um die Tests aufzurufen, wird die PHPUnit Binary im `vendor`-Verzeichnis in der Kommandozeile mit dem Parameter `-c` und der zuvor erstellten XML-Konfigurationsdatei aufgerufen:

```
1 $ vendor/bin/phpunit -c phpunit.xml
```

Dies startet die vier Tests und zeigt folgende Ausgabe an:

PHPUnit 13.2.1 by Sebastian Bergmann and contributors.

Runtime: PHP 8.5.7

Configuration: /var/www/html/wet2vo-examples/vo12/phpunit/phpunit.xml

.... 4 / 4 (100%)

Time: 00:00.025, Memory: 18.00 MB

OK (4 tests, 4 assertions)

In diesem Fall wurden alle Tests in 25 Millisekunden durchgeführt und bestanden.

Schließlich kann auch noch ein sogenannter Code-Coverage Report erstellt werden, der überprüft, wie viel vom Quellcode mit Unit Tests abgedeckt wurde. Dieser lässt sich wie folgt erzeugen:

```
1 $ vendor/bin/phpunit -c phpunit.xml --coverage-html coverage
```

Abbildung 12.7 zeigt das Ergebnis, das als HTML-Bericht im Ordner `coverage` vorliegt.

Da hierfür im Hintergrund Xdebug verwendet wird, muss dieses als Modul geladen und den Code-Coverage-Modus aktiviert haben.

12.4.5 Weitere Unit Tests

Um weitere konkrete Unit Tests zu sehen und sich daran Beispiele zu nehmen, reicht es, das `tests`-Verzeichnis der meisten Composer-Komponenten zu durchforsten. Diese enthalten in der Regel Unit Tests, die auch selbst ausgeführt werden können.

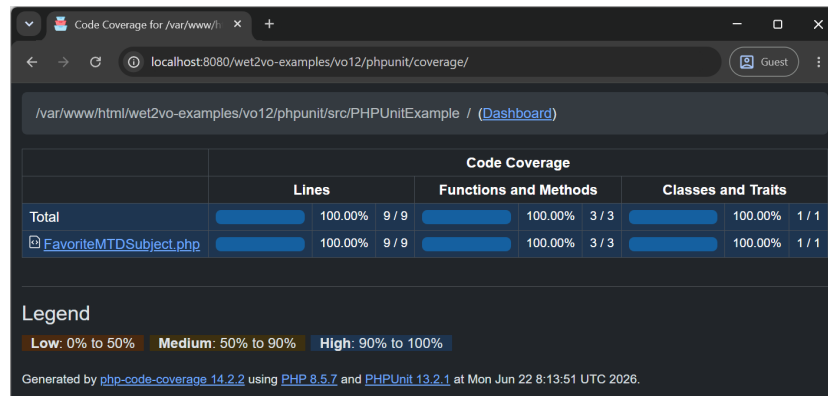


Abbildung 12.7: 100 % des Quellcodes sind durch Unit Tests abgedeckt.

12.4.6 Funktionale Tests

Unit Tests prüfen einzelne Bausteine isoliert. Um sicherzustellen, dass die Applikation auch im Zusammenspiel korrekt arbeitet, kommen *funktionale Tests* (auch Integrations- oder HTTP-Tests genannt) zum Einsatz. Bei einer Slim-Applikation lässt sich dies elegant umsetzen, da das App-Objekt selbst einen Request entgegennimmt und eine Response zurückliefert, ohne dass ein echter Webserver laufen muss.

Das Prinzip ist einfach: Die Applikation wird wie im Produktivbetrieb aufgebaut (Container, Routen, Middleware), anschließend wird ein `ServerRequest`-Objekt erzeugt und über `$app->handle($request)` verarbeitet. Das Ergebnis ist ein Response-Objekt, dessen Status-Code, Header oder Inhalt überprüft werden können.

```

1 <?php
2
3 namespace SlimExampleTest;
4
5 use PHPUnit\Framework\TestCase;
6 use Slim\Factory\AppFactory;
7 use Slim\Psr7\Factory\ServerRequestFactory;
8
9 final class HomePageTest extends TestCase
10 {
11     public function testHomePageShowsForm(): void
12     {
13         $app = AppFactory::create();
14         // Routen und Middleware wie in der Applikation registrieren ...
15
16         $request = (new ServerRequestFactory())->createServerRequest("GET", "/");
17         $response = $app->handle($request);
18
19         $this->assertSame(200, $response->getStatusCode());
20         $this->assertStringContainsString("Tell Me Your Name", (string) $response->
            getBody());
21     }
22 }
```

Damit Test und Produktivcode dieselbe Konfiguration verwenden, wird das Aufset-

zen der Applikation in der Praxis in eine gemeinsame Datei (etwa `app▸bootstrap.php`) ausgelagert, die sowohl von `public▸index.php` als auch von den Tests eingebunden wird. So wird die Applikation nur an einer Stelle definiert und im Test exakt so geprüft, wie sie später ausgeführt wird.

12.4.7 Test-Alternative: Pest

Mit *Pest*¹⁸ hat sich in den letzten Jahren eine populäre Alternative zu PHPUnit etabliert. Das Ziel von Pest ist es, Tests leserlicher und einfacher zu schreiben. Das Framework baut intern auf PHPUnit auf (die aktuelle Version Pest 4.7.3 nutzt PHPUnit 12) und benötigt mindestens PHP 8.3. Bestehende PHPUnit-Tests können weiterhin parallel ausgeführt werden. Pest lässt sich mit wenigen Schritten installieren und starten:

```
1 $ composer require pestphp/pest --dev --with-all-dependencies
2 $ ./vendor/bin/pest --init
3 $ ./vendor/bin/pest
```

Diese drei Zeilen installieren, initialisieren und starten Pest mit einem vorkonfigurierten Beispielttest.

Funktions-basierte Syntax

Anstelle von Testklassen mit Methoden setzt Pest auf einzelne, mit `test()` (oder dem Alias `it()`) definierte Funktionen. Der folgende Test verdeutlicht die Grundstruktur:

```
1 test("example", function () {
2     expect(true)->toBeTrue();
3 });
```

Mit der `expect()`-Anweisung wird ein Wert übergeben, der untersucht werden soll. Daran werden *Expectations* wie `toBeTrue()` oder `toBe()` angehängt, die überprüfen, ob der Wert dem erwarteten entspricht. Diese flüssig lesbare Schreibweise (*fluent API*) ist das Kernmerkmal von Pest.

Die Unit Tests in Pest

Die zuvor mit PHPUnit erstellten Tests der Klasse `FavoriteMTDSubject` lassen sich in Pest deutlich kompakter formulieren. Die `say()`-Methode etwa:

```
1 test("say returns the favorite subject", function () {
2     subject = new FavoriteMTDSubject("Web Technologies");
3     expect($subject->say())->toBe("The best subject in MTD is Web Technologies!");
4 });
```

Für das Testen von Exceptions bietet Pest die angehängte Methode `throws()`, was den Test gegenüber PHPUnit weiter verkürzt:

```
1 test("constructor rejects an empty subject", function () {
2     new FavoriteMTDSubject("");
3 })->throws(InvalidArgumentException::class);
```

¹⁸<https://pestphp.com/>

Darüber hinaus bietet Pest fortgeschrittene Funktionen wie *Datasets* (das Ausführen desselben Tests mit verschiedenen Eingabewerten), *Architecture Testing* (das Überprüfen von Architekturregeln) sowie seit Pest 4 auch *Browser Testing* für End-to-End-Tests im Browser. Weitere Beispiele liefert die Pest-Dokumentation¹⁹.

12.5 Weiterführende Literatur

Literatur zu Microframeworks ist eher spärlich vorhanden. In der Regel sind die offiziellen Dokumentationen der beste Ausgangspunkt, es gibt aber mit [6–8] drei eBooks zum Slim Framework 4, die von Daniel Opitz, einem der Core-Entwickler des Frameworks stammen. Wer eine Slim-Anwendung entwickelt, findet hier praktische Snippets und Artikel für die verschiedensten Erweiterungen und Szenarien.

Das Thema Unit Tests bzw. Testing ist sehr gut in [9] beschrieben. Die ausführlichste Ressource ist jedoch mit Sicherheit [1], wenngleich sie bereits in die Jahre gekommen ist und nicht mehr die neuesten Versionen abdeckt. Da das Buch jedoch vom Autor von PHPUnit verfasst ist, ist es eine gute Ressource im Umgang mit Testing und Test-Driven Development. Für aktuelle Versionen empfiehlt sich ergänzend das stets gepflegte Online-Handbuch von PHPUnit²⁰.

Das modernste Buch, das Testen mit PHP sehr holistisch abdeckt ist jedoch sicherlich [4]. Es ist sehr leserlich geschrieben, der Autor ist sehr aktiv in der PHP-Community und beantwortet auf Social Media auch gerne Fragen. Mit [3] existiert auch noch eine fortgeschrittene Version, die spezifisch auf Test-Tools eingeht.

¹⁹ <https://pestphp.com/docs/>

²⁰ <https://docs.phpunit.de/>

Quellenverzeichnis

- [1] Sebastian Bergmann. *PHPUnit Pocket Guide. Test-Driven Development in PHP*. Sebastopol, USA: O'Reilly, 8. Nov. 2005 (siehe S. 12-28).
- [2] Nicholas J. Giannini. „Vulnerable Web Application Framework“. Masterarbeit. Kingston, RI, USA: University of Rhode Island, Jan. 2015. URL: <https://digitalcommons.uri.edu/theses/629/> (siehe S. 12-2).
- [3] Chris Hartjes. *The Complementary PHP Testing Tools Cookbook*. php[architect], 9. Nov. 2021 (siehe S. 12-28).
- [4] Chris Hartjes. *The Grumpy Programmer's Guide To Testing PHP Applications*. php[architect], 26. Mai 2020 (siehe S. 12-28).
- [5] Josh Lockhart u. a. *Slim Middleware*. 9. Juli 2024. URL: <https://www.slimframework.com/docs/v4/concepts/middleware.html> (siehe S. 12-14).
- [6] Daniel Opitz. *Slim 4 – eBook*. Bd. 1. Eigenveröffentlichung, 12. Aug. 2023. URL: <https://ko-fi.com/dopitz/shop> (siehe S. 12-28).
- [7] Daniel Opitz. *Slim 4 – eBook*. Bd. 2. Eigenveröffentlichung, 12. Aug. 2023. URL: <https://ko-fi.com/dopitz/shop> (siehe S. 12-28).
- [8] Daniel Opitz. *Slim 4 – eBook*. Bd. 3. Eigenveröffentlichung, 12. Aug. 2023. URL: <https://ko-fi.com/dopitz/shop> (siehe S. 12-28).
- [9] Rainier Sarabia. *Test-Driven Development with PHP 8. Build extensible, reliable, and maintainable enterprise-level applications using TDD and BDD with PHP*. Birmingham, Großbritannien: Packt Publishing, 13. Jan. 2023 (siehe S. 12-21, 12-28).